CHAPTER 3

Introduction to MPI

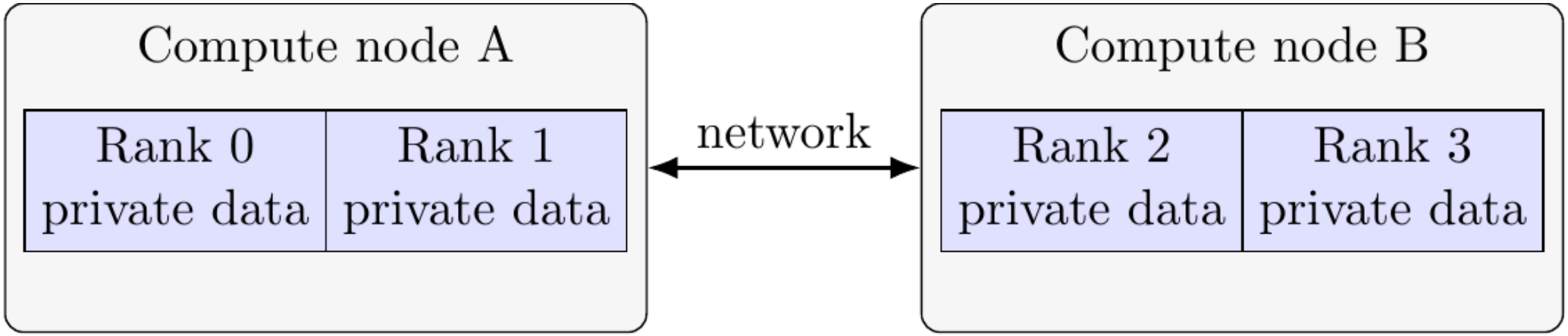
Python + mpi4py • Open MPI • message passing

Parallel and Distributed Computing

What you will be able to do

- Explain ranks, communicators, and SPMD execution.
- Install and verify an MPI environment on your platform.
- Use mpi4py for object and numerical buffer messages.
- Build, run, and validate a parallel sum.

Four ranks can live on two nodes



Each rank has private ordinary variables, even when ranks share a node.

Rank numbers identify processes

Term	Meaning
Process	Running instance of the program
Core	Hardware execution resource
Node	Computer with processors and memory
Rank	Process ID within a communicator
Communicator	Group plus separate message context

A rank is meaningful only inside its communicator.

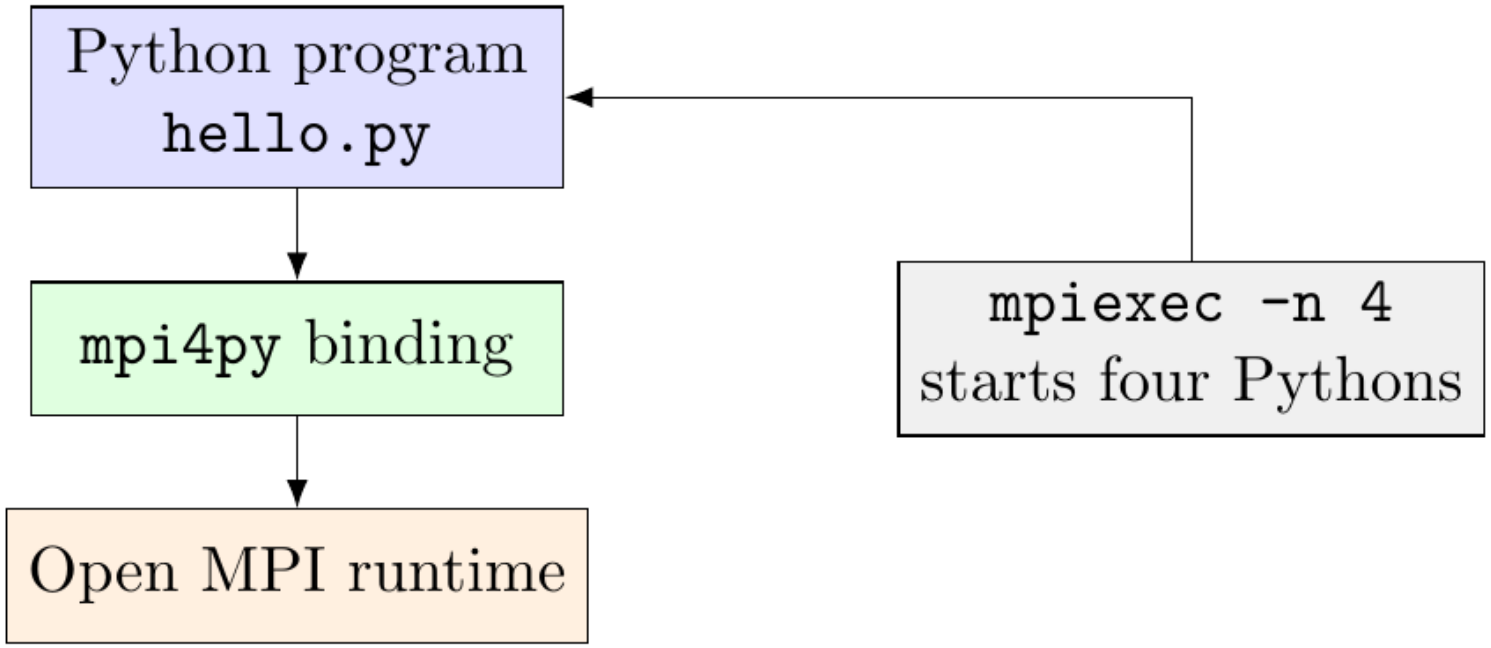
One program, different branches

```
value = 10
if rank == 0:
    value = 99
print(f"Rank {rank}: {value}", flush=True)
```

Rank 0 → 99
Rank 1 → 10
Rank 2 → 10

The assignment on rank 0 does not change another rank's variable.

Python sits above the MPI runtime



mpiexec starts multiple Python interpreters; each imports mpi4py.

Choose one consistent install route

Platform	Recommended route	Terminal
Ubuntu / Debian	Open MPI packages + venv	Linux shell
macOS	Homebrew Open MPI + venv	Terminal
Windows	Ubuntu under WSL 2	Ubuntu shell
Windows native	Microsoft MPI + venv	PowerShell

Keep the launcher, MPI library, Python, and mpi4py compatible.

Install Open MPI on Ubuntu

```
sudo apt update
sudo apt install python3 python3-venv python3-dev \
   build-essential openmpi-bin libopenmpi-dev
python3 -m venv .venv
source .venv/bin/activate
python -m pip install mpi4py numpy
```

Use the Ubuntu terminal for these commands, including in WSL.

Install Open MPI on macOS

```
brew install python open-mpi
python3 -m venv .venv
source .venv/bin/activate
python -m pip install mpi4py numpy
mpiexec --version
```

Install Homebrew first from brew.sh; use matching Python and Homebrew architectures.

Use Ubuntu under WSL for Windows

```
# Administrator PowerShell
wsl --install -d Ubuntu

# After restart: open Ubuntu
# Follow the Ubuntu installation commands

# Verify WSL version in PowerShell
wsl --list --verbose
```

Keep project files and Python inside the Ubuntu environment.

Native Windows uses Microsoft MPI

- Install x64 Python and the MS-MPI runtime.
- Create a venv with py -3 -m venv .venv.
- Install mpi4py and NumPy with the venv Python.
- Launch with MS-MPI’s mpiexec and that same Python.

Native MS-MPI is a separate route from WSL Open MPI.

Create a native Windows Python environment

```
py -3 -m venv .venv
$py = ".\.venv\Scripts\python.exe"
& $py -m pip install mpi4py numpy
where.exe mpiexec
& $py -c "from mpi4py import MPI; print(MPI.Get_library_version())"
mpiexec -n 4 $py hello.py
```

The reported library should be Microsoft MPI.

Check syntax, then launch ranks

```
python -m py_compile hello.py
mpiexec -n 4 python hello.py

# Runner that aborts the job on unhandled error
mpiexec -n 4 python -m mpi4py hello.py
```

py_compile checks syntax; mpiexec starts the parallel program.

A complete hello program

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
host = MPI.Get_processor_name()
print(f"Rank {rank} of {size} on {host}", flush=True)
if rank == 0:
    print(f"Launched {size} processes", flush=True)
```

The hello output can arrive in any order

```
Rank 2 of 4 on node
Rank 0 of 4 on node
Launched 4 processes
Rank 3 of 4 on node
Rank 1 of 4 on node
```

- Four lines identify four separate processes.
- Rank 0 prints the process count.
- flush=True does not order output globally.

Launch every supplied Python example

```
mpiexec -n 4 python -m mpi4py hello.py
mpiexec -n 2 python -m mpi4py messages.py
mpiexec -n 2 python -m mpi4py ping_pong.py
mpiexec -n 4 python -m mpi4py token_ring.py
mpiexec -n 4 python -m mpi4py parallel_sum.py 17
```

Run from the examples/chapter3 directory with the environment active.

mpi4py has two messaging interfaces

Property	Python objects	NumPy buffers
Calls	send / recv	Send / Recv
Transfer	Serialized objects	Typed numerical memory
Receive	Returns a Python object	Fills preallocated array
Best for	Metadata and control	Large numeric arrays

Use the same interface at both ends of a message.

Object messages keep Python types

```
if rank == 0:
    payload = {"text": "MPI", "values": [1, 2, 3]}
    comm.send(payload, dest=1, tag=11)
elif rank == 1:
    payload = comm.recv(source=0, tag=11)
    print(payload["text"], payload["values"])
```

The receiver gets a reconstructed Python dictionary.

Buffer receives need enough space

```
# Rank 0
data = np.array([1.5, 2.5, 3.5], dtype=np.float64)
comm.Send([data, MPI.DOUBLE], dest=1, tag=10)

# Rank 1
data = np.empty(8, dtype=np.float64)
status = MPI.Status()
comm.Recv([data, MPI.DOUBLE], source=0, tag=10,
          status=status)
count = status.Get_count(MPI.DOUBLE)
print(data[:count])
```

Messages match by envelope

Field	Example	Purpose
Communicator	MPI.COMM_WORLD	Separates communication contexts
Source	rank 0	Who sent the message
Destination	rank 1	Who receives it
Tag	10	Distinguishes message purpose

Datatype compatibility and buffer capacity are checked after selection.

Status reveals what arrived

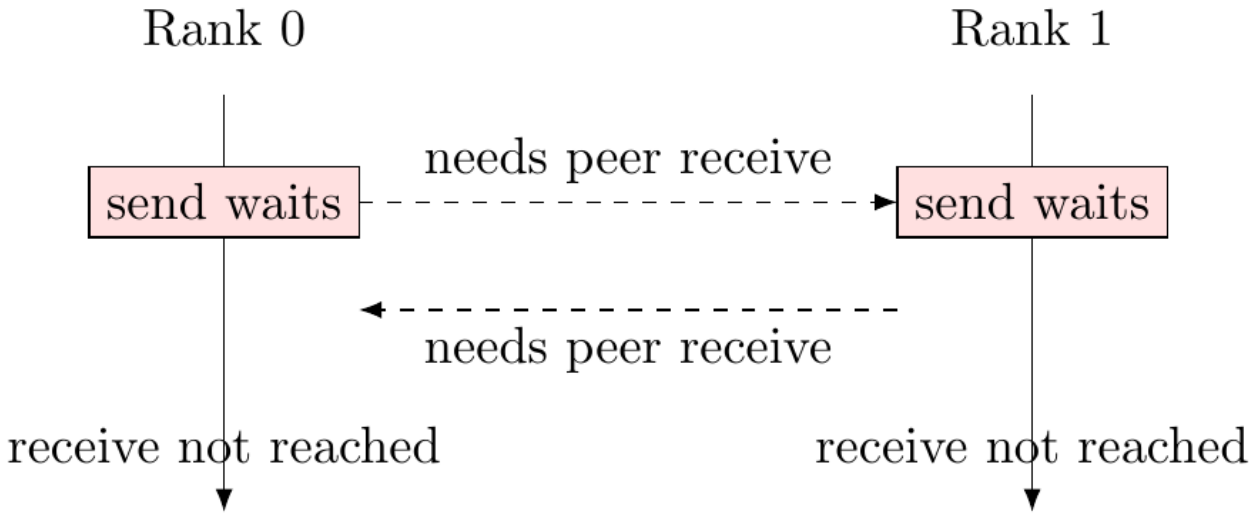
```
status = MPI.Status()
comm.Recv([buffer, MPI.DOUBLE],
          source=MPI.ANY_SOURCE, tag=10,
          status=status)
source = status.Get_source()
count = status.Get_count(MPI.DOUBLE)
```

Wildcard source can accept whichever worker finishes first.

Blocking calls have local meanings

- A blocking send returns when its send buffer may be reused.
- A blocking receive returns when data is available locally.
- Send completion does not prove the receiver processed the data.
- Programs must not depend on hidden MPI buffering.

Symmetric send-first code can deadlock



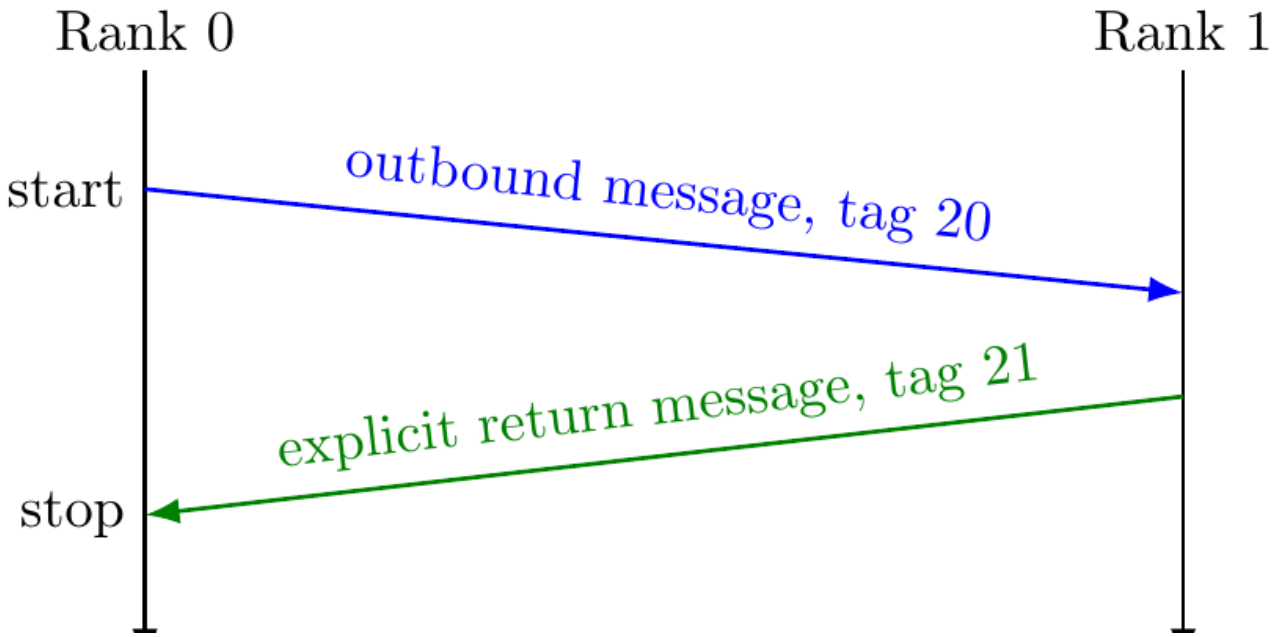
A small message may seem to work; a larger one can expose the cycle.

Sendrecv breaks the exchange cycle

```
peer = 1 - rank
outgoing = np.array([rank], dtype=np.float64)
incoming = np.empty_like(outgoing)
comm.Sendrecv([outgoing, MPI.DOUBLE],
              dest=peer, sendtag=9,
              recvbuf=[incoming, MPI.DOUBLE],
              source=peer, recvtag=9)
```

Use separate send and receive buffers with matching peers and tags.

Ping-pong measures a round trip



Take both timestamps on rank 0; rank clocks need not be synchronized.

Repeat timings before drawing conclusions

100

warm-up exchanges

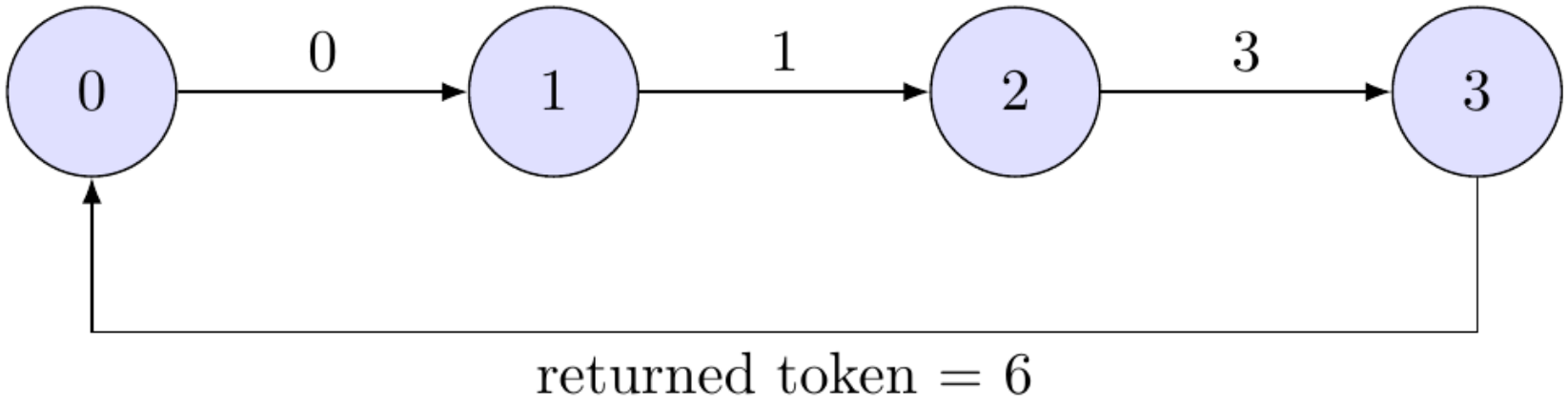
1,000

timed round trips

mean = elapsed time ÷ repetitions

These are experiment settings from ping_pong.py, not a universal latency.

A token ring passes one value onward



For four ranks, the returned token is 0 + 1 + 2 + 3 = 6.

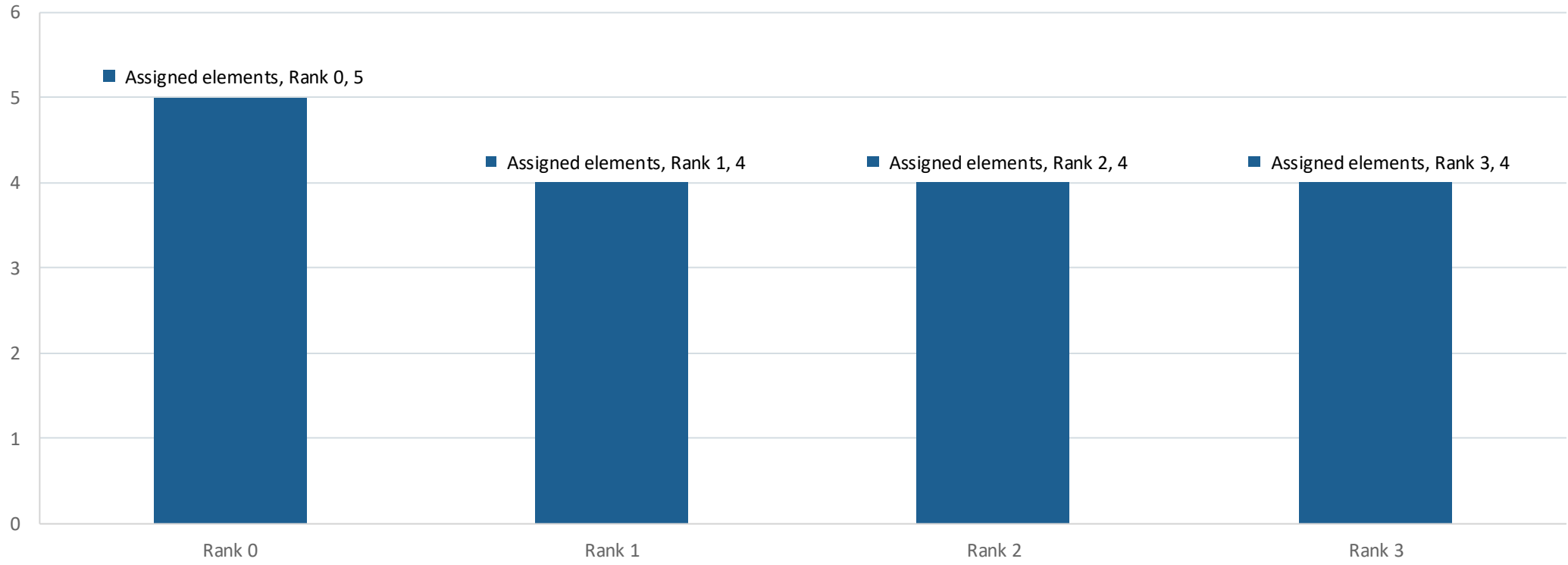
Block decomposition gives contiguous work

q = N // P

m = N % P

- First m ranks get q + 1 elements.
- Remaining ranks get q elements.
- Every element belongs to exactly one half-open range.

17 elements split as 5, 4, 4, 4



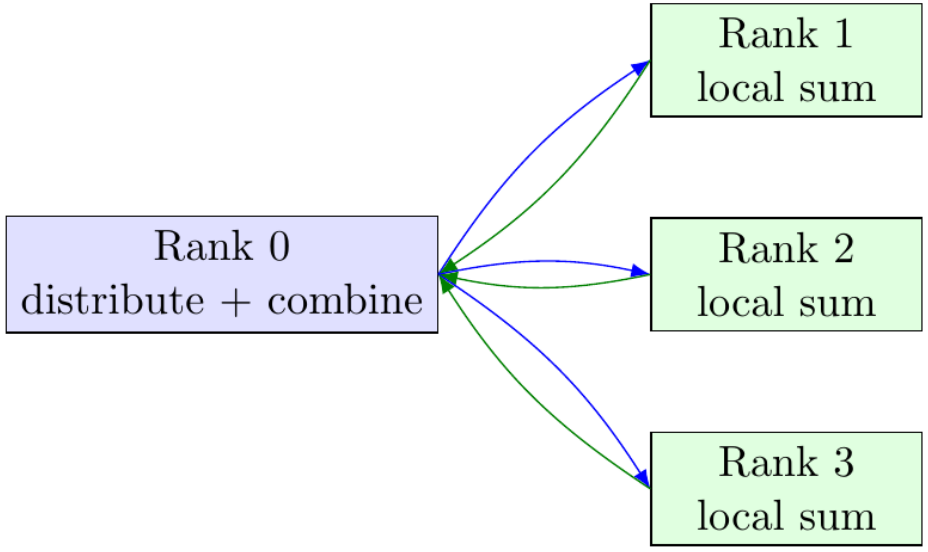
Analytical work counts for N = 17, P = 4; these are not measured runtimes.

Empty partitions are valid

N	P	Block counts	What to check
17	4	5, 4, 4, 4	Uneven split
3	4	1, 1, 1, 0	One empty block
0	4	0, 0, 0, 0	All blocks empty

An empty list is still a valid serialized Python object message.

Rank 0 distributes and combines



Blue: data blocks
Green: partial sums

Rank 0 also sums its own block.

Data blocks use tag 40; partial sums use tag 41; validation uses tag 42.

Parallel sum starts with balanced blocks

```
def block(n, processes, rank):
    q, remainder = divmod(n, processes)
    count = q + (rank < remainder)
    start = rank * q + min(rank, remainder)
    return start, count

start, count = block(n, size, rank)
local = data[start:start + count]
```

The half-open slice contains exactly count values.

Each rank computes, then rank 0 checks

```
partial = sum(local)
if rank == 0:
    total = partial
    for worker in range(1, size):
        total += comm.recv(source=worker, tag=41)
    formula = n * (n + 1) // 2
    passed = (total == serial == formula)
else:
    comm.send(partial, dest=0, tag=41)
```

The answer stays 153 across rank counts

N	Ranks	Expected sum	Purpose
17	1	153	Sequential baseline
17	2	153	Two processes
17	4	153	Uneven blocks
3	4	6	Empty partition
0	4	0	All partitions empty

Also compare against the serial sum and N(N + 1) / 2.

Use the symptom to find the first check

Symptom	Likely cause	First check
Hang	Unmatched message or cyclic wait	Trace peer, tag, call order
Truncation	Receive buffer too small	Compare datatype and count
Wrong total	Partition bug	Check ranges and local sums
All sizes are 1	Mixed MPI environments	Match launcher and library

Practice with one, two, and four ranks

- Run hello.py and explain output order.
- Run messages.py and inspect source, tag, and count.
- Trace ping-pong and token-ring message order.
- Validate parallel_sum.py for N = 17, 3, and 0.
- Modify the sum to compute a maximum.

Build a correct MPI program in stages

- Choose a consistent MPI and Python environment.
- Define each message by communicator, peer, tag, and data type.
- Use an ordering that cannot wait in a cycle.
- Partition every element once, including empty blocks.
- Validate with small known answers and multiple rank counts.

Source: Chapter 3, Introduction to MPI • mpi4py official documentation